



دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

درس معماری سیستم های قابل باز پیکربندی

دکتر صاحب الزمانی

شایان نقی زاده

۴۰۲۱۳۱۰۴۳

تمرین سری اول

سوال ۱

با ذکر دلیل بیان کنید جمالت زیر صحیح هستند یاخیر

- در یک پروژه با زمان محدود بهترین راه جهت پیاده سازی الگوریتم پردازشی استفاده از تراشه های قابل بازپیکربندی است.

نادرست

اگر زمان محدود باشد سریع ترین راه استفاده از پردازنده های عام منظوره است یعنی آن را به صورت نرم افزاری پیاده سازی کنیم چون پیاده سازی با استفاده از fpga نیاز به دانش سخت افزاری و بهبود هایی است که زمان بیشتری نیاز دارد ولی اگر بین asic و fpga باشیم و زمان محدود باشد fpga بهتر است.

- طراحی های مبتنی بر پردازنده های همه منظوره و تراشه های خاص منظوره، دو انتهای بردار کارایی و انعطاف پذیری هستند.

درست

با این فرض که ترتیب آمده شده مورد نظر نباشد چون پردازنده های عام منظوره بیشترین انعطاف پذیری و خاص منظوره بیشترین کارایی را دارند که در سوال برعکس آمده است. اگر مهم نباشد این ترتیب بله هر دو دو سر بردار کارایی و انعطاف پذیری هستند.

- معماری قابل بازپیکربندی جهت حل مشکل دسترسی حافظه در کامپیوتر فون نویم ارائه شده است.

نادرست

درست است که در با این معماری ما می توانیم مشکل دسترسی به حافظه را کم کنیم ولی این معماری برای ایجاد کردن نمونه اولیه قبل تولید و قابلیت پیکربندی مجدد آن معرفی شد و پیش از ظهور FPGA ها و معماری های کاملاً قابل بازپیکربندی، معماری های domain-specific و application-specific مانند ASIC ها و DSP ها وجود داشتند که برای کاربردهای خاص طراحی شده بودند و به کاهش مشکلات مربوط به گلوگاه حافظه کمک می کردند. این معماری ها، به واسطه طراحی اختصاصی، عملکرد بهینه تری در انجام وظایف مشخص ارائه می دادند و تراشه های قابل برنامه پذیر اصولاً برای خاصیت پیکربندی آن ها معرفی شدند.

- در کاربردهای فضایی و محیط های دارای تشعشعات زیاد، تراشه های مبتنی بر FLASH بهترین گزینه انتخابی هستند.

نادرست

بهترین گزینه برای soft error آنتی فیوز ها هستند. البته با فرض اینکه این سوال درباره تراشه های قابل پیکربندی است نه حافظه ها

- از تراشه های مبتنی بر آنتی فیوز به دلیل مقاومت مناسب در برابر دمای بالا در کاربردهای صنعتی استفاده می شود.

درست

از این تراشه فقط به دلیل مقاومت دمایی بالا نیست که در صنعت استفاده می شود بلکه به دلیل کم مصرف بودن از نظر توان

مساحت کم تر داشتن نسبت به بقیه و دیر تر دچار ageing شدن و عدم نیاز به حافظه خارجی در صنعت استفاده می کنند. می توان اینگونه گفت که یکی از دلایل استفاده می تواند این باشد ولی دلیل اصلی این نیست.

- تراشه های CGRA با دارا بودن واحدهای خاص منظوره بیشتر، توان کمتری نسبت به FPGA ها دارند.

درست

تراشه های CGRA به دلیل اینکه بخشی های دارند که به صورت LUT و با آن ریز دانگی پیاده سازی نشده توان مصرفی کمتری دارند چون بهینه تر پیاده سازی شدند.

- استفاده از FPGA ها در مقایسه با تولید یک تراشه خاص باعث کاهش هزینه تولید محصول خواهد شد.

نادرست

با توجه به اینکه در تولید انبوه، هزینه ساخت یک تراشه خاص (ASIC) به دلیل سرشکن شدن هزینه ها بر تعداد زیاد، می تواند از استفاده از FPGA ها کمتر باشد، ولی در مورد یک تراشه مشخصا استفاده از fpga ارزان تر است.

- یک ASIC همواره سریعتر از یک FPGA دستورات پردازشی سطح بالا را انجام خواهد داد.

درست

تراشه های ASIC (Application-Specific Integrated Circuit) به طور خاص برای انجام یک وظیفه یا مجموعه ای از وظایف بهینه سازی و طراحی شده اند و به همین دلیل، برای اجرای دستورات پردازشی سطح بالا بسیار سریع تر از FPGA ها عمل می کنند . FPGA ها از ساختار قابل برنامه ریزی استفاده می کنند که اگرچه انعطاف پذیری بیشتری دارد، اما به اندازه ی یک ASIC اختصاصی و بهینه نیست و همین باعث کاهش سرعت نسبی آن ها می شود.

- افزایش تعداد ورودی یک LUT همواره باعث افزایش سرعت مدار پیاده سازی شده با استفاده از آن خواهد شد.

نادرست

افزایش تعداد ورودی های یک LUT (Look-Up Table) لزوماً باعث افزایش سرعت مدار نمی شود. با افزایش تعداد ورودی ها، پیچیدگی و زمان لازم برای جستجو در جدول نیز بیشتر می شود که می تواند باعث کاهش سرعت شود. بنابراین، افزایش تعداد ورودی ها می تواند سرعت را کاهش دهد یا حتی در برخی موارد بهینه سازی را مختل کند این طور می توان گفت که استفاده از چند سطح LUT یا چند LUT به طور موازی می تواند سرعت بهتری نسبت به یک LUT بزرگ داشته باشد.

- بلوک های UltraRAM در کنار بلوک های DSP برای پیاده سازی الگوریتم های هوش مصنوعی به کمک FPGA خانواده Zynq بسیار مناسب هستند.

درست

بلوک های UltraRAM همراه با بلوک های DSP برای اجرای الگوریتم های هوش مصنوعی در FPGA های خانواده Zynq بسیار مناسب هستند. این بلوک ها از توان پردازش با بازده بالا که برای کاربردهای هوش مصنوعی ضروری است، پشتیبانی می کنند. تحقیقات نشان می دهد که بلوک های DSP در FPGA ها می توانند عملیات های ضرب و جمع متوالی (MAC) را که پایه و اساس

محاسبات یادگیری عمیق و استنتاج مدل‌های هوش مصنوعی هستند، به‌طور مؤثری پردازش کنند. به عنوان مثال، DSPها با پیکربندی‌های بهینه حافظه مانند UltraRAM، امکان پشتیبانی از عملیات‌های کم دقت و مسیرهای داده‌ی سفارشی را فراهم می‌کنند که برای استنتاج یادگیری عمیق بسیار مفید است.

سوال ۲

در یک سیستم ایمنی مرتبط با خودرو نیاز به طراحی یک سیستم ایمنی با قابلیت اطمینان بالا می‌باشد که بایستی دارای امکان به روزرسانی الگوریتم ایمنی نیز باشد. همچنین زمان عملکرد سیستم نیز بایستی به صورت time-Real Hard باشد. برای طراحی این سیستم در صورت نمونه سازی و در صورتی که ۱ میلیون نسخه از آن نیاز باشد استفاده از چه نوع بستر پردازشی را پیشنهاد می‌نمایید؟ برای انجام محاسبات، هزینه‌های مربوط به ساخت معماری پیشنهادی خود را از اینترنت استخراج نمایید؟

در واقع در ابتدا به نظر می‌رسد ما ۳ گزینه بیشتر نداریم fpgaهای صنعتی fpgaهای مبتنی بر SoC و در آخر میکروکنترلرها که در ادامه بحث می‌کنیم

۱. FPGAهای صنعتی

مزایا: FPGAها به دلیل معماری قابل پیکربندی، انعطاف‌پذیری بالایی دارند و به‌خوبی برای سیستم‌های real-time مناسب هستند. به دلیل اجرای عملیات به‌صورت سخت‌افزاری، زمان تأخیر را می‌توان بسیار پایین نگه داشت و برای کاربردهایی با پردازش موازی بالا که در سیستم‌های ایمنی خودرو بسیار حائز اهمیت است، بسیار مناسب هستند.

معایب: در FPGAهای ساده که تنها fabric قابل پیکربندی دارند، اجرای نرم‌افزار برای به‌روزرسانی الگوریتم‌ها دشوار است. همچنین، هزینه‌های اولیه و زمان توسعه در FPGAها بالا هستند و استفاده از آنها نیازمند تجربه و تخصص ویژه‌ای است.

۲. FPGAهای مبتنی بر SoC (System on Chip)

مزایا: FPGAهای دارای SoC علاوه بر بخش منطقی، دارای پردازنده‌های تعبیه‌شده هستند معمولاً ARM Cortex این معماری امکان به‌روزرسانی الگوریتم‌ها را فراهم می‌کند و می‌توان ترکیب مناسبی از پردازش نرم‌افزاری و سخت‌افزاری را ارائه داد. این گزینه در سیستم‌هایی که نیاز به real-time سخت دارند و به‌روزرسانی نرم‌افزاری نیاز است، بسیار مناسب است.

معایب: قیمت اولیه این قطعات نسبت به FPGAهای ساده بالاتر است. اما در حجم تولید بالا، این هزینه‌ها کاهش می‌یابد و قابل رقابت می‌شود.

۳. MCU یا میکروکنترلرها

مزایا: میکروکنترلرهای ایمنی که دارای گواهی‌های مربوط به سیستم‌های ایمنی خودرو هستند مثل خانواده Aurix از Infineon، معمولاً به‌طور خاص برای کاربردهای ایمنی طراحی شده‌اند و هزینه پایین‌تری نسبت به FPGAها دارند. این پردازنده‌ها قابلیت اجرای کدهای به‌روزرسانی‌شده را دارند و برای پردازش نرم‌افزاری زمان-واقعی (به صورت نسبی) مناسب هستند.

معایب: اگرچه MCU ها قابلیت real-time دارند، توانایی رقابت با FPGA ها در زمان بندی سخت افزاری را ندارند، به ویژه در کاربردهای پیچیده ایمنی که نیازمند پردازش موازی هستند.

قیمت: بسته به مدل و قابلیت های ایمنی، MCU های ایمنی خودرو حدوداً ۵ تا ۵۰ دلار قیمت دارند و در حجم بالا قیمت کاهش می یابد.

از آن جایی که در صورت سوال گفته شده قابلیت بروز رسانی ما نمی توانیم به سمت asic ها برویم درسته از نظر اقتصادی مقرون به صرفه هستند ولی قابل پیکربندی مجدد نیستند گزینه بعدی استفاده از یک پردازنده یا میکرو کنترلر است ولی چون نیاز کاربرد حساس است و ما نیاز به سیستم سریع داریم گزینه خوبی نیست پس باید به سمت fpga ها برویم ولی چون تعداد ما برای تولید زیاد است باید سمت fpga های ارزون تر برویم سمتی ما نیاز به بروز رسانی داریم که در این کاربرد fpga های مبتنی بر SoC که دارای واحد پردازشی هستند می توانند کار را بهتر جلو ببرند زیرا به روز رسانی الگوریتم ها از طریق پردازنده های تعبیه شده امکان پذیر است و همچنین پاسخ گویی real-time و پردازش موازی در fabric منطقی FPGA می تواند کارایی مورد نیاز را فراهم کند. و در مقایسه با FPGA های بدون SoC، انعطاف پذیری و سازگاری بیشتری با استانداردهای ایمنی خودرو مانند ISO ۲۶۲۶۲ دارد.

نتیجه گیری و پیشنهاد

با توجه به الزامات پروژه و تولید ۱ میلیون نسخه، FPGA مبتنی بر SoC گزینه مناسبی است، چرا که:

- به روز رسانی الگوریتم ها از طریق پردازنده های تعبیه شده امکان پذیر است.
 - پاسخ گویی real-time سخت و پردازش موازی در fabric منطقی FPGA می تواند کارایی مورد نیاز را فراهم کند.
 - در مقایسه با FPGA های بدون SoC، انعطاف پذیری و سازگاری بیشتری با استانداردهای ایمنی خودرو
- در عین حال، اگر پیچیدگی و الزامات پردازشی پایین تر باشد، ممکن است میکروکنترلرهای ایمنی نیز بتوانند نیازهای سیستم را برآورده کنند و هزینه ها را به طور قابل ملاحظه ای کاهش دهند.

از نظر قیمتی FPGA ها و FPGA های مبتنی بر SoC قیمت های متفاوتی دارند و بستگی به منابع آن ها است که دامنه های متفاوتی دارند که می توانیم تهیه کنیم ولی از آن جایی که کاربرد ما پیچیدگی زیادی ندارد نیاز به تراشه خیلی پیچیده نخواهیم داشت و از سمتی چون تعداد تراشه که ما می خواهیم سفارش بدهیم زیاد است شرکت های ساخت تخفیف های بیشتر نسبت به خرید تکی به ما می دهند که در این حالت می تواند خیلی به نفع ما باشد طبق قیمت های [سایت](#) قیمت ارزان ترین تراشه قابل بازپیکربندی که در آن یک پردازنده تک هسته Cortex-A9 دارد ۵۸ دلار آمریکا است که تراشه (ZYNQV۰۰۰-XCVZ۰۰VS) است ولی این قیمت قیمت تک تراشه است و همانطور که گفته شد در تعداد بالا حتی ممکن است خیلی ارزان تر در نظر گرفته شود این نوع تراشه مناسب کاربرد ما می باشد برای اندازه گیری هزینه ساخت و طراحی از آن جایی که این تراشه از قبل طراحی شده است و نیاز به طراحی ندارد هزینه NRE برای ما در نظر گرفته نمی شود (البته با فرض اینکه ما هزینه توصیف و ابزارهای نرم افزاری و تست نداشته باشیم) و فقط هزینه ساخت تراشه برای تعداد که می خواهیم حساب می شود.

هزینه ساخت برابر است با تعداد تراشه در قیمت هر تراشه به اضافه NRE که برای آن تراشه است

هزینه ساخت و طراحی ما

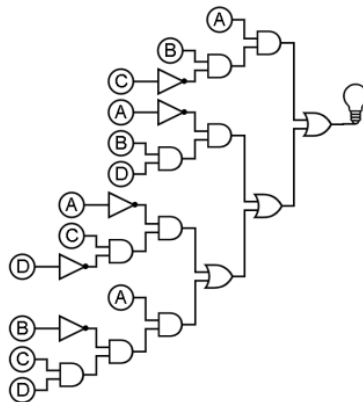
$$1000000 * 58 + \text{NRE} = 58000000$$

همانطور گفته شد این هزینه کران بالای هزینه ما است و ممکن است خیلی پایین تر از این باشد.

و در صورت استفاده از میکروکنترلر های هزینه در حد چند میلیون خواهد بود چون باز هم NRE در این حالت صفر است.

سوال ۳

میخواهیم مدار زیر را یک بار با LUT های ۳ ورودی و بار دیگر با LUT های ۴ ورودی پیاده سازی کنیم به طوری که در هر حالت تعداد LUT های مورد اس تفاده کمینه باشد.



A	B	C	D	ABC	$\bar{A}BD$	$\bar{A}C\bar{D}$	$ABCD$	F
•	•	•	•	•	•	•	•	•
•	•	•	∖	•	•	•	•	•
•	•	∖	•	•	•	∖	•	∖
•	•	∖	∖	•	•	•	•	•
•	∖	•	•	•	•	•	•	•
•	∖	•	∖	•	∖	•	•	∖
•	∖	∖	•	•	•	∖	•	∖
•	∖	∖	∖	•	∖	•	•	∖
∖	•	•	•	•	•	•	•	•
∖	•	•	∖	•	•	•	•	•
∖	•	∖	•	•	•	•	•	•
∖	•	∖	∖	•	•	•	∖	∖
∖	∖	•	•	∖	•	•	•	∖
∖	∖	•	∖	∖	•	•	•	∖
∖	∖	∖	•	•	•	•	•	•
∖	∖	∖	∖	•	•	•	•	•

حالت اول LUT های چهار متغیره

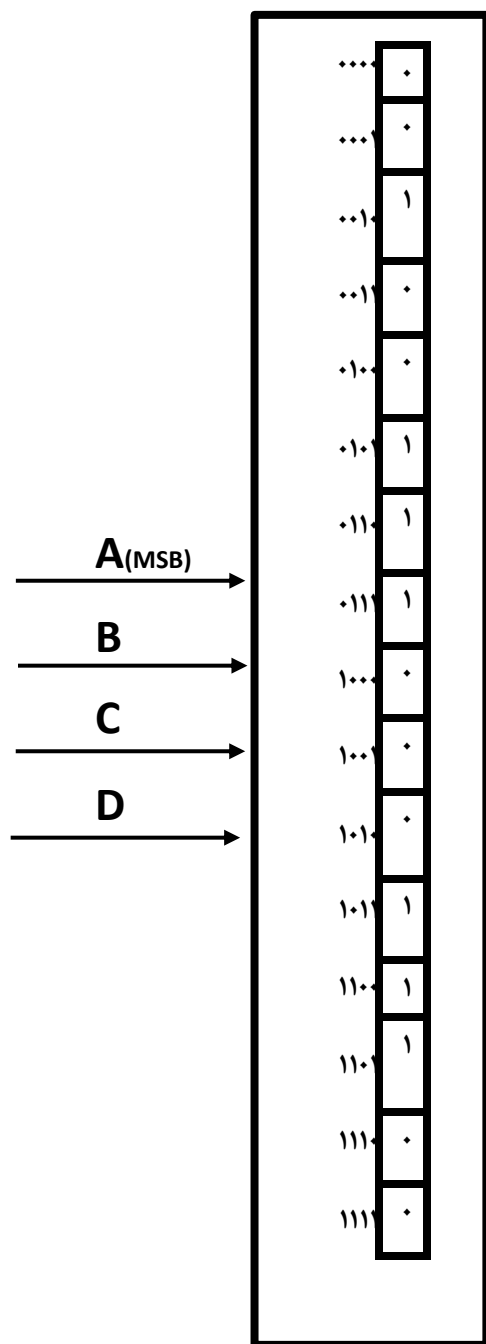
برای ساخت تابع با LUT چهار متغیره به ترکیب مقدار متغیرها به عنوان آدرس خانه ها نگاه می کنیم و حاصل تابع را در آن خانه قرار می دهیم و چون ما ۴ متغیر داریم ۱۶ حالت متغیر داریم پس نیاز به ۱۶ خانه حافظه داریم که مقدار هر خانه متناسب است با مقدار تابع با توجه به آن متغیرها

(مثال)

حاصل

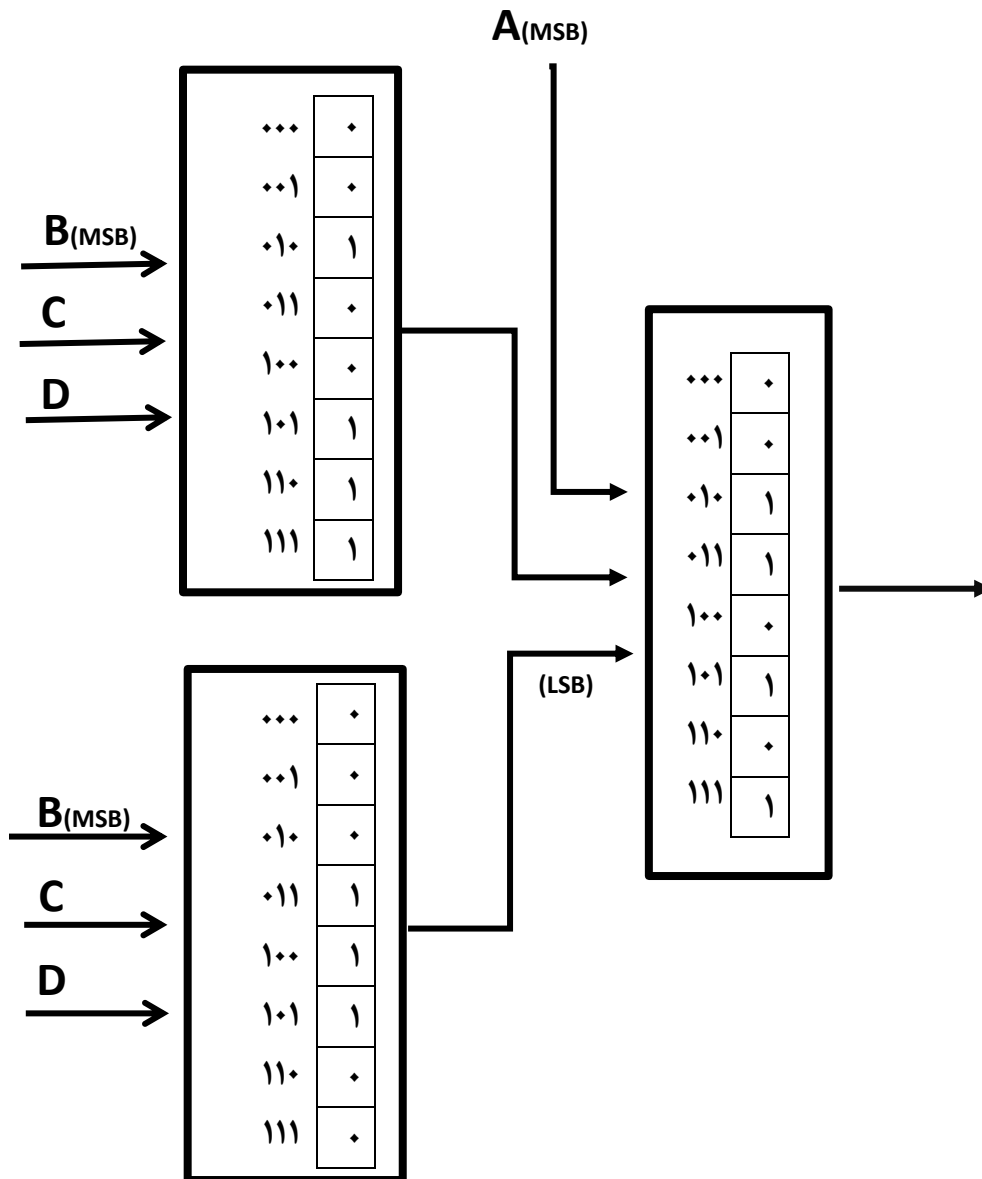
$$A=0, B=0, C=0, D=1$$

می شود صفر که اگر به آدرس ۰۰۰۱ نگاه کنیم در این خانه مقدار صفر ذخیره شده است و به همین ترتیب برای بقیه حالت ها خانه های حافظه پر می شود.

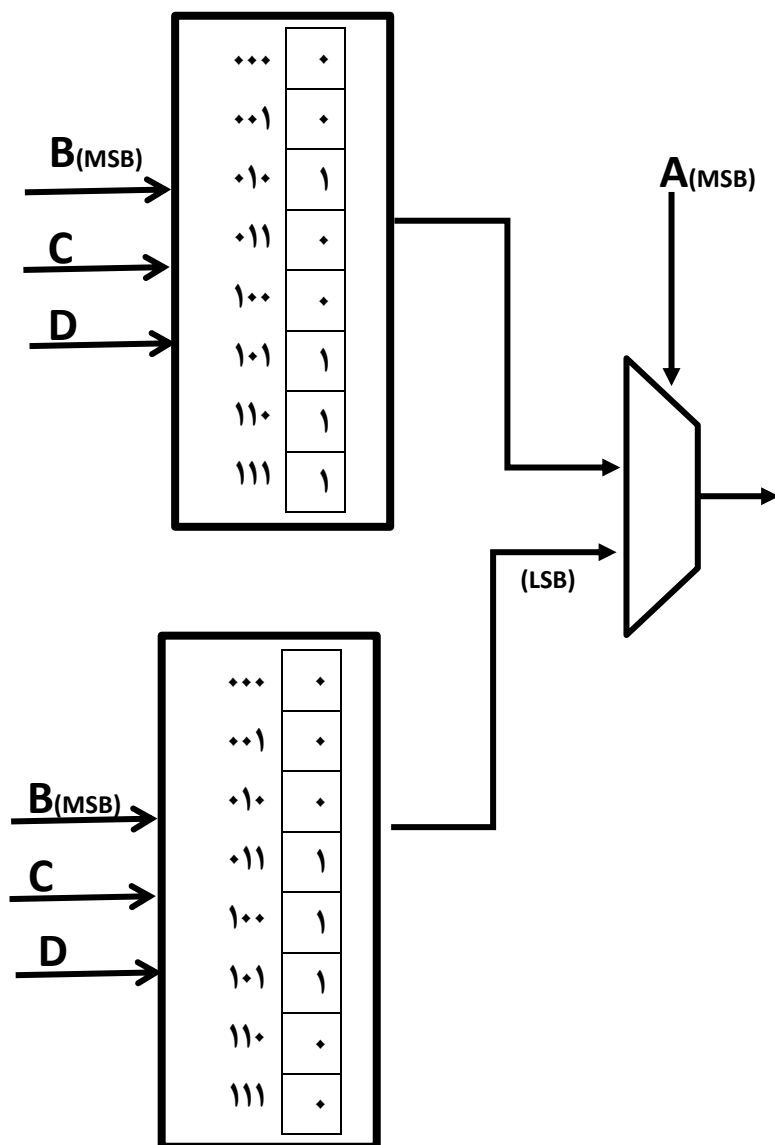


حالت دوم LUT های ۳ متغیره

برای پیاده سازی با LUT های ۳ متغیره کافی است با توجه به متغیره پرارزش جدول را بشکنیم و خط آدرس A را به LUT سطح دوم بدهیم و با توجه به خروجی که از سطح اول به دست می آوریم و مقدار A سطح دوم را کامل کنیم در واقع A همان کار SELECTOR در مالتی پلکسر را انجام می دهد.



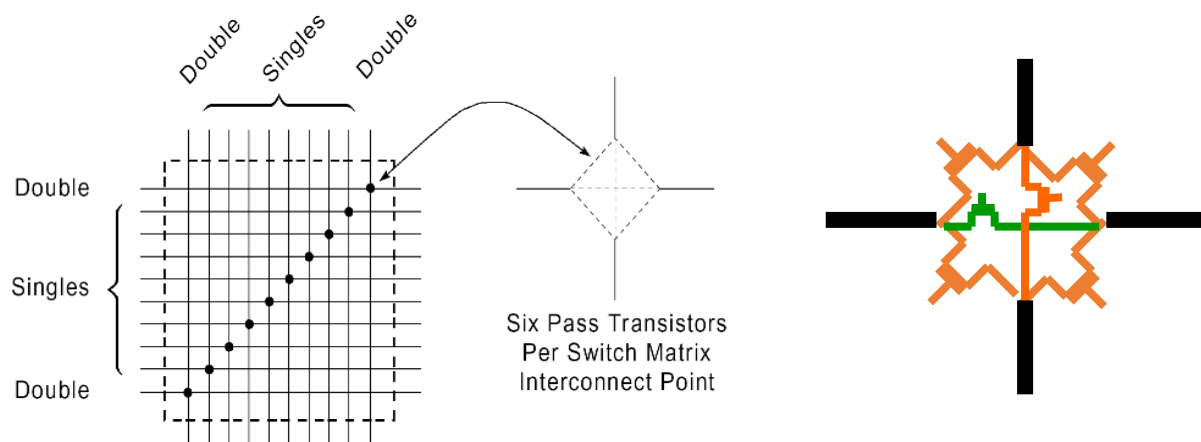
*راه حل دوم استفاده از مالتی پلکسر در سطح دوم است که ممکن است به خاطر نداشتن مالتی پلکسر نتوانیم پیاده سازی کنیم و مانند بالا از یک LUT دیگر استفاده کنیم.



سوال ۴

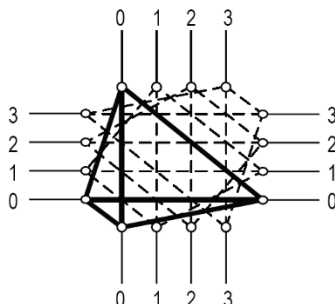
معماری سویچ های Wilton و Disjoint را توضیح داده و میزان Fs را در هر یک گزارش نمایید . آیا معماری دیگری برای اتصال سویچ ها می شناسید؟

قبل صحبت درباره سویچ باید به این توجه کنیم که هر programmable switch matrix (PSM) از ماتریسی از شش pass transistor تشکیل شده که ما با برنامه ریزی سلول SRAM آن می توانیم آن ارتباط را برقرار کنیم.



همانطور که در شکل مشخص است ما می توانیم با برنامه ریزی مناسب این سلول های حافظه ارتباط را ایجاد کنیم ولی تمام سویچ های موجود در تراشه ها مختلف تمام این شش ترانزیستور را ندارند و متناسب با این تعداد معماری های متفاوت سویچ های متفاوت که در ادامه توضیح می دهیم شکل گرفته شد در حقیقت این تعداد ترانزیستور برای ایجاد ارتباط در یک سویچ ماتریس مناسب همه کاربرد ها نبود و برای همین شروع به ایجاد معماری های متفاوت شد تا بتوانیم منابع را به درستی مصرف کنیم.

معماری سوئیچ‌های disjoint

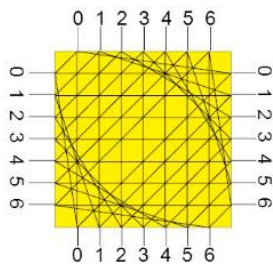


سوئیچ چهار وروی با معماری disjoint و $f_s = 3$

در این معماری هر سیم ورودی به سوئیچ تنها به سیم‌های هم شماره خود متصل می‌شود به طور مثال در شکل سیم صفر تنها به سیم‌های صفر دیگر متصل شده و ارتباط را برقرار می‌کند

همانطور که مشخص است در این معماری مسیرهای ما به خاطر این ویژگی محدود می‌شود و ممکن است مسیرهای بهینه را از دست بدهیم. به دلیل این تفکیک، گزینه‌های کمتری برای مسیریابی وجود دارد، چرا که یک سیگنال نمی‌تواند به تمامی مسیرهای ممکن دسترسی داشته باشد. این ساختار ممکن است انعطاف‌پذیری مسیریابی را کاهش دهد، اما در عین حال از پیچیدگی جلوگیری می‌کند.

در شکل بالا که ۴ سیم از هر طرف وارد یک سوئیچ می‌شود با توجه به معماری disjoint ما به $f_s = 3$ می‌رسیم که یعنی ما می‌توانیم همزمان به ۳ سیم دیگر متصل شویم و اتصال برقرار کنیم.



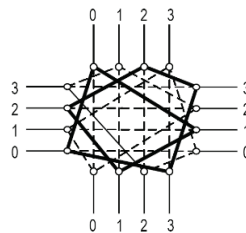
مثال - سوئیچ هفت وروی با معماری disjoint و $f_s = 3$

معماری سوئیچ‌های wilton

برای رفع مشکل سوئیچ‌های disjoint به سمت معماری wilton می‌رویم که با همان تعداد سوئیچ به سمت رفع محدودیت آن‌ها یعنی محدودیت مسیریابی ناشی از اتصالات هم نام می‌رویم.

همانطور که به ذهن می‌رسد راه حل عوض کردن اتصالات است یعنی هر سیم بتواند به تعدادی سیم ناهم‌نام و هم نام ولی نه همه متصل شود که انعطاف پذیری بیشتری در مسیریابی به ما بدهد در این حالت محدودیت مسیریابی که دچار می‌شدیم را بهبود می‌بخشیم این مزیت باعث می‌شود دامنه اتصالات ما مانند معماری disjoint محدود نباشد و احتمال مسیریابی بهینه را برای ما بیشتر می‌کند در مقایسه با حالت قبلی که disjoint بوده است این سوئیچ انعطاف پذیری بهتری دارد.

در واقع سوئیچ Wilton با تغییرات جایگشتی منظم، سیگنال‌ها را در FPGA به مسیرهای پراکنده هدایت می‌کند تا از تراکم و تداخل جلوگیری شود و کارایی مسیریابی افزایش یابد.



سوئیچ چهار وروی با معماری Wilton و $f_s = 3$

در این سوئیچ هم مثل سوئیچ disjoint ما $f_s = 3$ است.

نتیجه گیری

Disjoint

این سوئیچ ساده تر است ولی دامنه محدودی به ما جهت مسیریابی می‌دهد و حتی ممکن است نتوانیم مسیر بهینه را پیاده کنیم و برای طرح‌های دارای پیچیدگی مشکل ساز است.

Wilton

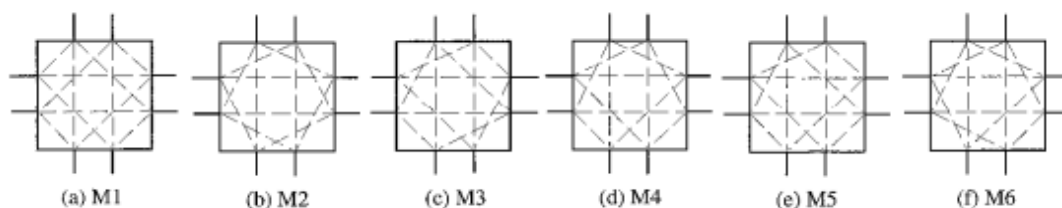
این سوئیچ از جهت اتصالات متفاوت و غیر هم نام دامنه بهتری به ما می‌دهد که برای طرح پیچیده بهینه است.

پس ما با این دو نوع معماری فقط می‌توانیم بعضی سلول‌های حافظه برنامه ریزی کنیم و بقیه اتصالات از قبل ایجاد شده و قابل پیکربندی نیستند که Wilton به ما دامنه بهتری را برای ارتباط می‌دهد.

معماری سوئیچ‌های Universal

نوع دیگری از سوئیچ‌ها که متوان حدس زد این است که هر کدام از این مسیرهای بتوانند به هر مسیری که می‌خواهد به اصطلاح بتوان هر دو نقطه متصل شود که به آن **universal** گفته می‌شود یعنی در این حالت تمام سلول‌های حافظه قابل مقدار دهی و پیکربندی هستند که می‌توان اینجوری گفت که هر از هر سمت قابل برنامه ریزی و متصل شدن به بقیه سیم‌ها در جهات مختلف است.

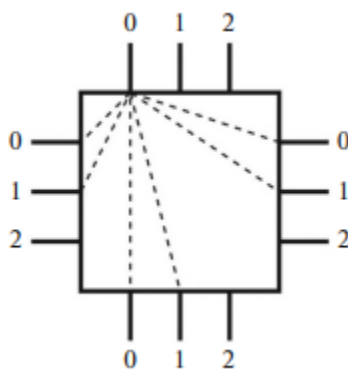
به طور مثال در شکل تمام حالت در صورت این که ورودی ما دو سیم باشد و از چهار جهت در یک سوئیچ **universal** که قابل برنامه ریزی است آمده است.



که در سوئیچ **universal** هم $f_s = 3$ است.

به طور کلی در این سه سوئیچ ما $f_s = 3$ است چون هر سیم ورودی می‌تواند از 3 سمت دیگر خارج شود و در این معماری‌ها بیشتر از یک اتصال در یک سمت نیست.

البته این معماری‌هایی که ما بررسی کردیم $f_s = 3$ دارد و می‌تواند بیشتر از این باشد حتی می‌تواند به تمام سیم‌ها حتی اگر در یک سمت باشد مسیر ایجاد شود ولی این کار باعث زیاد شدن تراکم سیگنال و ایجاد مشکل می‌شود ولی هر f_s بیشتر داشته باشیم انعطاف پذیری بیشتری خواهیم داشت.



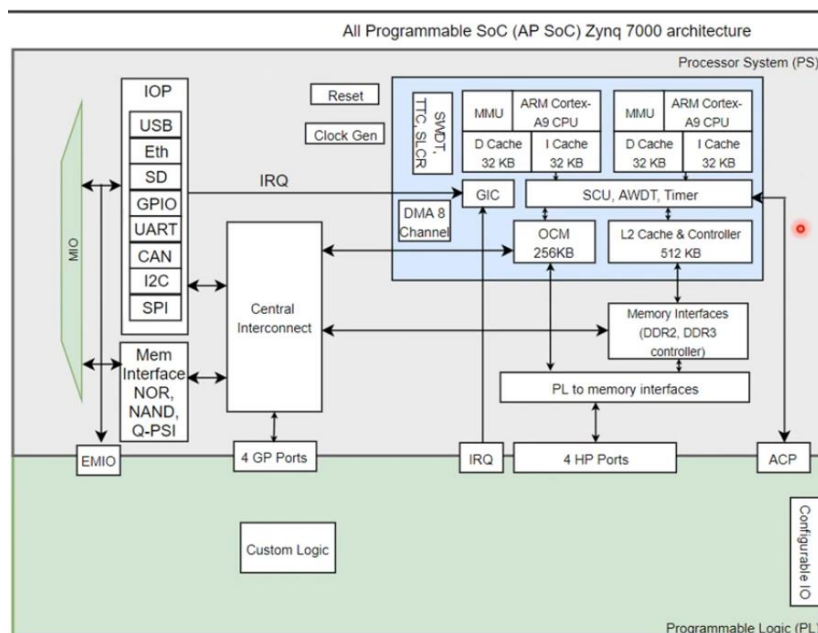
سوئیچ چهار ورودی با $f_s = 6$

سوال ۵

آشنایی اولیه با ابزار و یوآدو : از شرکت زایلینکس در این درس دانشجویان با استفاده از ابزار ویوآدو به انجام پروژه ها خواهند پرداخت. هدف از انجام پروژه ها، آشنایی عملی با طراحی توأم بر روی سیستم های قابل بازپیکربندی است. برای این منظور در این بخش در ابتدا دانشجویان میبایست نرم افزار ویوآدو را بر روی سیستم خود نصب کنند. سپس با بررسی لینک زیر در ارتباط با نحوه طراحی توأمان و نحوه کار با ابزار آشنایی لازم را کسب کرده و توضیحات موردنیاز را در ارتباط با این نوع طراحی ارائه دهند.

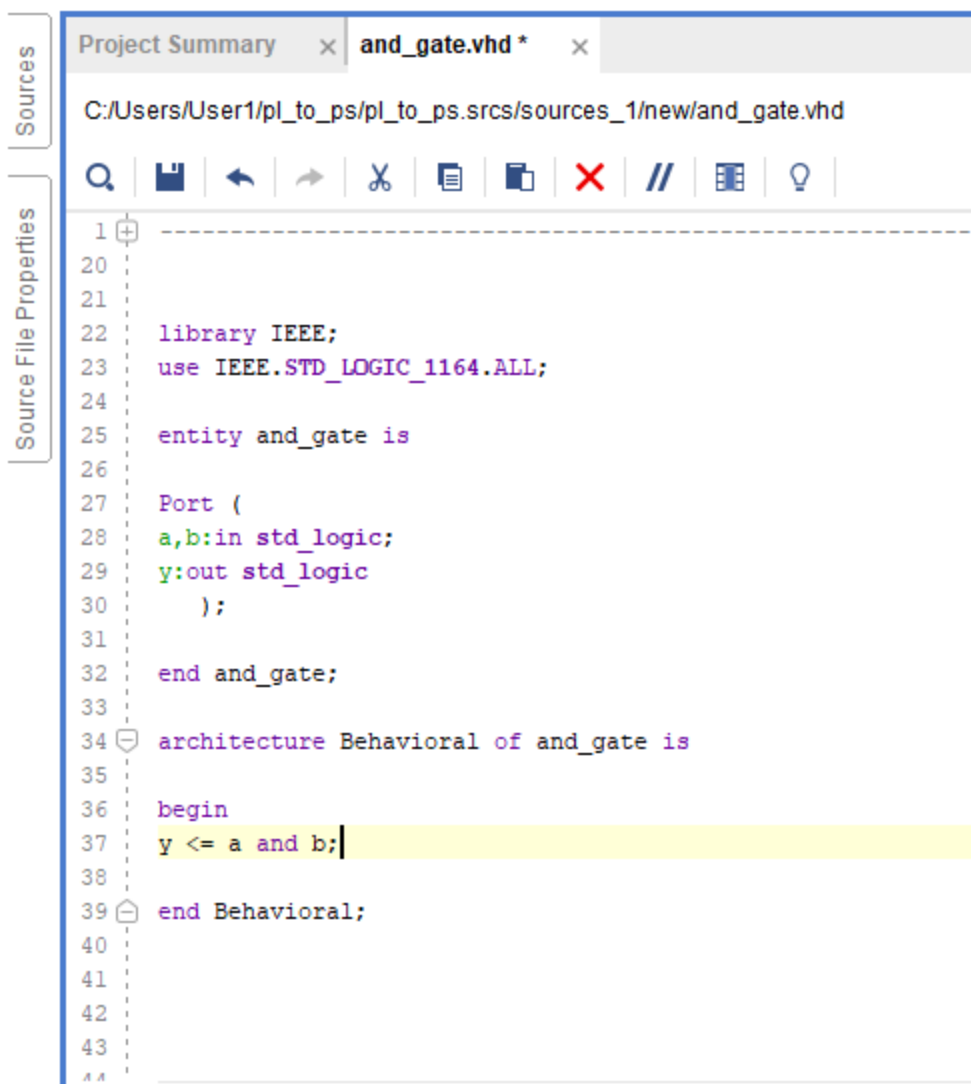
پاسخ ۵

۱. برای شروع یک پروژه ایجاد می کنیم و بورد مورد نظر را انتخاب می کنیم ما از بورد Artys ZV-10 استفاده می کنیم چون این بورد هم تراشه ZYNQV۰۰۰ هرو دارد یعنی یک پردازنده PS و یک قسمت قابل پیکربندی را دارد PL که ما می خواهیم اتصال این دو را برقرار کنیم به نحوی که قسمت Programmable Logic برای ما and را انجام دهد و خروجی and وارد Processing System ما شود و not را انجام دهد و خروجی را به ما نشان دهد. در حقیقت هدف ایجاد ارتباط بین این دو واحد برای این نوع fpga است.



شکل یک معماری سطح بالا از این تراشه را نشان می دهد که قسمت PL و PS آن و تمام ماژول های که در پردازنده آن وجود دارد مانند تعداد هسته که در شکل و تراشه ما دو است مقدار کش و حافظه و.. مشخص شده است.

۲. مرحله بعد ایجاد کد توصیف گیت AND است که برای این کار کافی است در قسمت Source یک فایل ایجاد کنیم و کد توصیف خود را به یکی از زبان های موجود اضافه کنیم (ما از vhdل استفاده کردیم).



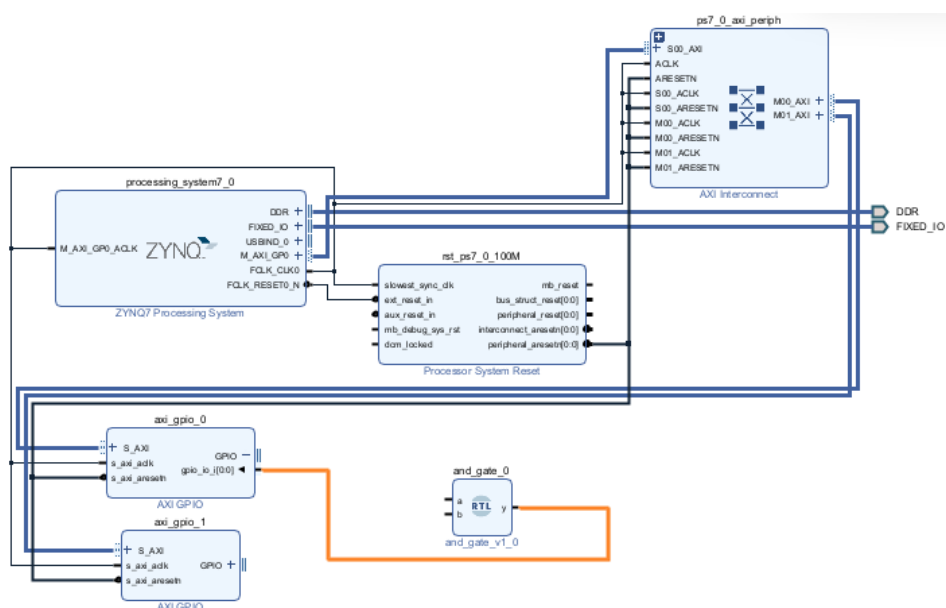
The screenshot shows a VHDL source file named 'and_gate.vhd' in a text editor. The left sidebar has tabs for 'Sources' and 'Source File Properties'. The code is as follows:

```
1  -----
20
21
22  library IEEE;
23  use IEEE.STD_LOGIC_1164.ALL;
24
25  entity and_gate is
26
27  Port (
28    a,b:in std_logic;
29    y:out std_logic
30  );
31
32  end and_gate;
33
34  architecture Behavioral of and_gate is
35
36  begin
37    y <= a and b;
38
39  end Behavioral;
40
41
42
43
```

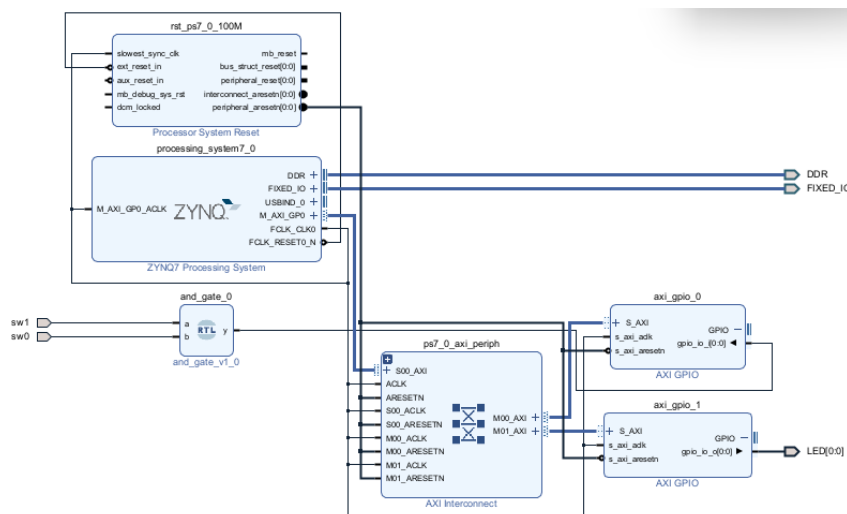
۳.

۴. در این مرحله فایل constraints را ایجاد می کنیم در واقع این فایل برای تخصیص پین های فیزیکی به سیگنال های طراحی که انجام می دهیم است از آن جایی که ما از بورد Artys ZV-۱۰ استفاده کردیم Constraint فایل آماده آن موجود می باشد و می توانیم دانلود کنیم و بخش های مورد نیاز را مانند ویدیو که LED و سویچ است را uncomment کنیم تا اتصالات آن برقرار شود همچنین برای راحتی می توانیم اسم پایه ها رو هم تغییر بدهیم.

۵. در مرحله بعد یک Design Block ایجاد می کنیم و گیت که با قسمت PL طراحی کردیم را اضافه می کنیم حالا برای ایجاد ارتباط با پردازنده نیاز به AXI-GPIO ها هستیم در واقع AXI GPIO یک ماژول قابل برنامه ریزی ورودی/خروجی است که با پروتکل AXI سازگار بوده و برای ارتباط آسان FPGA یا SoC با دستگاه های جانبی استفاده می شود. بعد از اضافه کردن و اتصالات را ایجاد می کنیم با استفاده از Automation که در ابزار موجود است و بعد نوع این GPIO ها رو مشخص میکنیم که برای GPIO۰ ورودی است و GPIO۱ خروجی چون GPIO۰ را به عنوان ورودی تعریف کردیم خروجی گیت AND را که توصیف کردیم به آن متصل می کنیم. و از GPIO۱ برای روشن کردن LED و گرفتن خروجی از قسمت PS استفاده می شود.



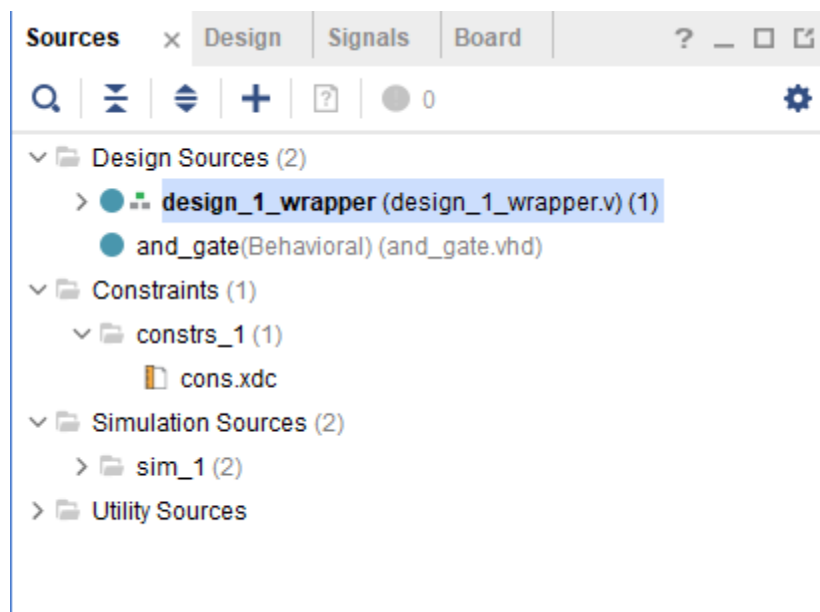
۶. حالا ترمینال خروجی و ورودی خود را با توجه به Constrains فایل خود ایجاد می کنیم در واقع با این کار سویچ ها رو به ورودی های AND و خروجی PS را به LED وصل می کنیم.



۷.

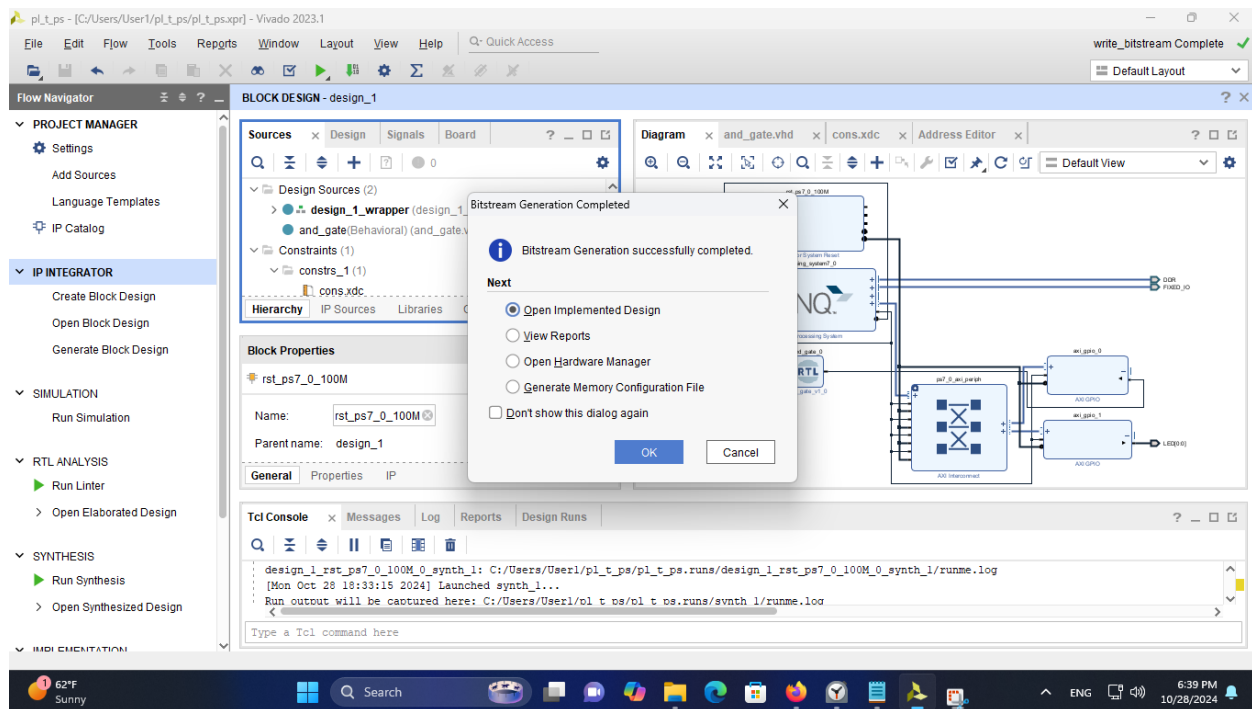
۸.

۹. یک Wrapper از طراحی ایجاد می کنیم که طراحی ما را شامل می شود.



۱۰.

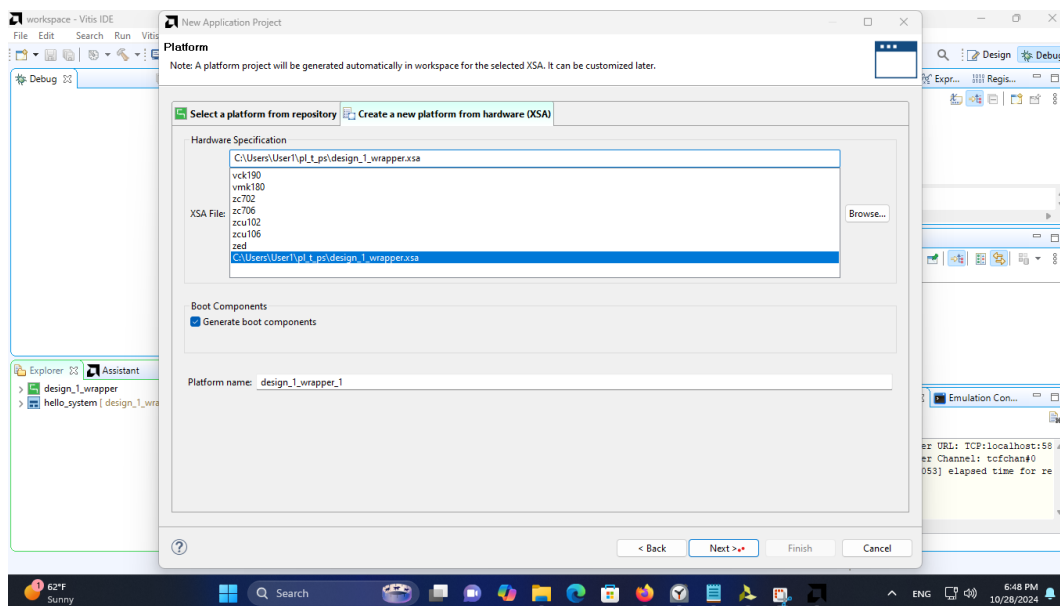
۱۱. Bitstream طراحی خود را ایجاد می کنیم.



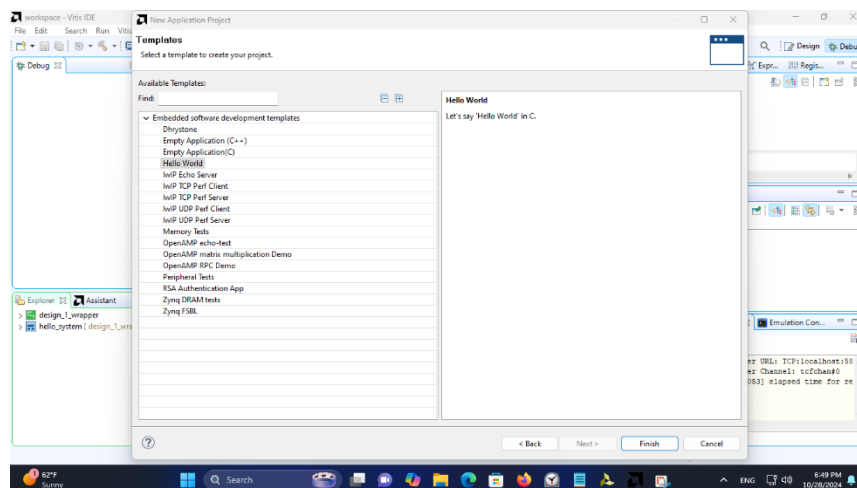
۱۲. از طراحی خود به همراه bitstream خروجی می گیریم.

۱۳. از آنجایی که نسخه ۲۰۲۳ دیگر SDK ندارد از Vitis استفاده می کنیم یک application project ایجاد می کنیم

۱۴. در هنگام ایجاد فایل .xsa را برای شناخت بستر به آن می دهیم.



۱۵. برای اینکه کتابخانه‌ها از قبل در سورس کد وجود داشته باشد برای template از برنامه hello word موجود استفاده می‌کنیم و آن را تغییر می‌دهیم



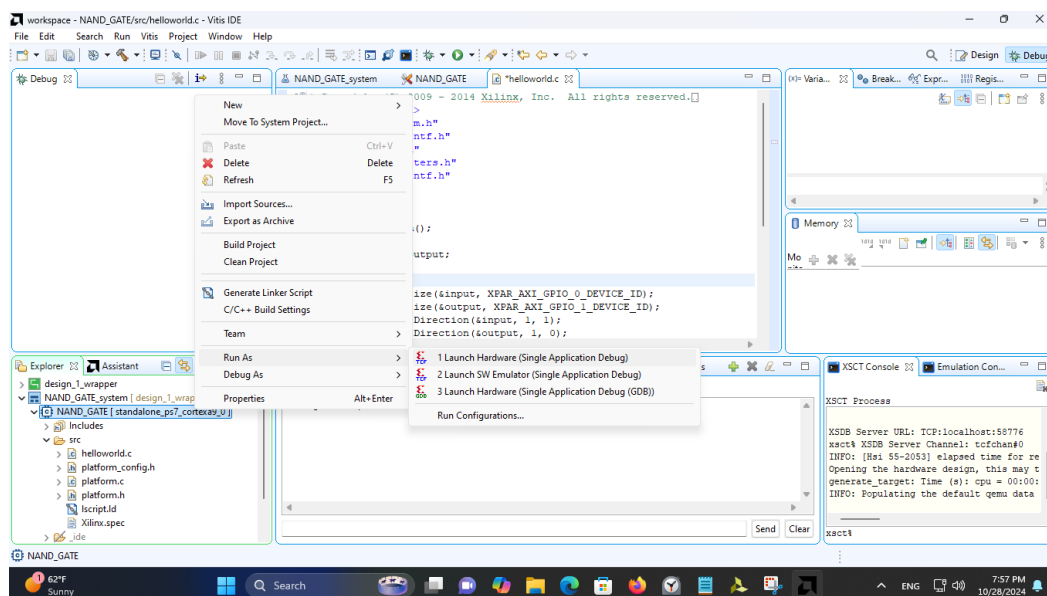
۱۶. در این مرحله سورس کد را متناسب با کاربرد خود ویرایش می‌کنیم کتابخانه xgpio.h و xparameters.h را اضافه می‌کنیم.

۱۷. کد را ویرایش می‌کنیم با توجه به اتصالات که کد ویرایش شده در پیوست وجود دارد

۱۸. البته در ویدیو برای تست درست پیکربندی شدن در کد یک پیام "we are up" که در صورت اجرای آن یک پیغام هم در کنسول دریافت خواهیم کرد.

۱۹. در این مرحله اتصال را از طریق uart به تراشه برقرار می‌کند تا بتواند پیغام را مشاهده کند

۲۰. در آخر برنامه رو روی تراشه اجرا می‌کنیم ولی قبلش باید تراشه را program کنیم تا قسمت PL هم درست کار کند. و در انتها برنامه را اجرا می‌کنیم



در حقیقت هدف این کار ایجاد ارتباط بین قسمت پردازنده (hard core) در تراشه ما با قسمت قابل پیکربندی است ما با ایجاد یک قسمت به صورت سخت افزاری (گیت and) و اتصال آن به پردازنده موجود از هر دو بخش استفاده کردیم و در انتها برنامه C خود را روی پردازنده اجرا کردیم که پردازنده برای انجام محاسبات and از قسمت سخت افزاری استفاده می کند و مقدار میانی را از آن می گیرد و به صورت نرم افزاری not می کند و با توجه به حاصل LED را کنترل می کند.